

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler dan Dasar Pemrograman

Samuel Hideaki Tangke Arung - 5024241062

2026

1 Pendahuluan

1.1 Latar Belakang

Praktikum sistem mikroprosesor dan sistem tertanam modul pertama ini membahas dasar pemrograman mikrokontroler modern 32-bit menggunakan STM32 Blackpill dan ESP32. Kedua mikrokontroler ini banyak digunakan dalam berbagai proyek industri saat ini. STM32 Blackpill yang berbasis prosesor ARM Cortex-M4 sering dipakai untuk aplikasi kendali presisi tinggi, sedangkan ESP32 yang memiliki dua inti prosesor dan konektivitas nirkabel sangat cocok untuk perangkat Internet of Things.

Untuk memprogram mikrokontroler ini, kita menggunakan Visual Studio Code yang dilengkapi dengan ekstensi PlatformIO. PlatformIO mempermudah pekerjaan karena kita dapat mengatur berbagai jenis board, compiler, dan library dalam satu tempat yang praktis lewat file konfigurasi `platformio.ini`.

Selain perangkat lunak, praktikan juga mempelajari sifat fisik dari pin masukan digital GPIO. Salah satu masalah yang sering ditemui adalah pin yang mengambang tanpa sambungan pasti, sehingga nilainya berubah-ubah secara tidak menentu. Hal ini dapat diatasi dengan memasang resistor pull-up atau pull-down. Praktikan juga mempelajari cara mengatasi getaran mekanis pada tombol sakelar yang disebut bouncing menggunakan metode debouncing, serta bagaimana mikrokontroler membaca tegangan analog menggunakan ADC untuk sensor seperti LDR. Pemahaman ini sangat penting sebagai bekal sebelum mempelajari sensor, motor, komunikasi serial, dan sistem kendali pada modul-modul berikutnya.

1.2 Dasar Teori

1.2.1 PlatformIO dan Visual Studio Code

PlatformIO adalah alat pengembangan embedded yang berjalan di dalam editor Visual Studio Code. PlatformIO membantu mengurus instalasi compiler, library, dan proses upload program ke board secara otomatis. Seluruh pengaturan proyek disimpan di dalam file `platformio.ini`. Di dalam file ini, kita dapat menentukan board yang digunakan, framework program, dan kecepatan serial monitor.

Struktur folder proyek PlatformIO terdiri dari:

- `src`: tempat meletakkan file kode utama program.
- `include`: tempat menyimpan file header tambahan jika diperlukan.
- `lib`: tempat menaruh library khusus yang kita buat sendiri.
- `.pio`: folder otomatis tempat menyimpan hasil compile program.

1.2.2 STM32 Blackpill dan Kerangka Kerja Arduino

STM32 Blackpill adalah board mikrokontroler berbasis chip STM32F401CCU6 dengan prosesor ARM Cortex-M4 32-bit. Chip ini berjalan pada kecepatan hingga 84 MHz, memiliki memori Flash 256 KB, dan RAM 64 KB. Board ini memiliki lampu LED bawaan pada pin PC13 yang menyala saat diberi logika LOW, serta satu tombol pengguna pada pin PA0.

Board Blackpill tidak memiliki chip USB-to-Serial di papannya. Port USB Type-C yang ada langsung terhubung ke pin USB internal mikrokontroler. Oleh sebab itu, untuk mengisi program dan melakukan debug, kita memerlukan alat tambahan bernama ST-Link V2. Alat ini terhubung ke mikrokontroler melalui jalur Serial Wire Debug yang menggunakan pin SWDIO dan SWCLK. Untuk mempermudah penulisan kode, STM32 dapat diprogram menggunakan framework Arduino STM32duino, sehingga kita tetap bisa menggunakan perintah sederhana seperti `pinMode` dan `digitalWrite`.

1.2.3 ESP32 dan Kerangka Kerja ESP-IDF

ESP32 adalah chip mikrokontroler 32-bit yang memiliki dua inti prosesor dan sudah dilengkapi dengan Wi-Fi serta Bluetooth. Board ESP32 DevKit umumnya sudah memiliki chip perantara USB-to-Serial bawaan, seperti CP2102 atau CH340, sehingga kita bisa langsung menghubungkannya ke komputer menggunakan kabel USB biasa tanpa alat tambahan.

ESP-IDF adalah framework resmi buatan Espressif yang berjalan di atas sistem operasi FreeRTOS. Berbeda dengan Arduino yang membagi alur kerja menjadi fungsi `setup` dan `loop`, ESP-IDF membagi program menjadi tugas-tugas yang berjalan mandiri dengan titik awal program bernama `app_main`. Framework ini memberikan kendali yang sangat mendalam terhadap perangkat keras dan efisiensi memori, sehingga sering dipakai pada produk industri.

1.2.4 Resistor Pull-Up dan Pull-Down

Pin masukan digital pada mikrokontroler memiliki hambatan masukan yang sangat besar. Jika sebuah pin tidak dihubungkan ke sumber tegangan maupun ke ground, pin tersebut berada dalam kondisi mengambang. Akibatnya, gelombang elektromagnetik atau derau listrik di sekitarnya dapat membuat pin membaca logika yang berubah-ubah antara HIGH dan LOW secara acak.

Untuk membuat status pin menjadi pasti, digunakan resistor pull-up atau pull-down:

1. **Pull-Up:** Pin GPIO dihubungkan ke tegangan 3.3V melalui resistor 10 kΩ, dan tombol dipasang di antara pin GPIO dengan ground. Saat tombol tidak ditekan, pin terbaca HIGH. Saat tombol ditekan, arus mengalir ke ground sehingga pin terbaca LOW.
2. **Pull-Down:** Pin GPIO dihubungkan ke ground melalui resistor 10 kΩ, dan tombol dipasang di antara pin GPIO dengan tegangan 3.3V. Saat tombol tidak ditekan, pin terbaca LOW. Saat tombol ditekan, pin terhubung ke tegangan sumber sehingga terbaca HIGH.

ESP32 memiliki resistor penarik internal yang bisa diaktifkan melalui program. Namun, pin khusus masukan seperti GPIO 34 sampai 39 tidak memiliki resistor internal, sehingga harus menggunakan resistor fisik dari luar.

1.2.5 Fenomena Bouncing dan Debouncing

Tombol mekanik bekerja menggunakan pelat logam lentur. Saat tombol ditekan atau dilepas, pelat logam tersebut tidak langsung menempel dengan sempurna, melainkan memantul beberapa kali dalam waktu yang sangat singkat sekitar 1 sampai 50 milidetik.

Karena prosesor mikrokontroler membaca sinyal dengan sangat cepat, pantulan sinyal sesaat ini dapat terbaca sebagai penekanan tombol berkali-kali. Cara untuk menyaring gangguan getaran ini disebut debouncing. Kita dapat mengatasinya melalui program dengan menunggu sampai sinyal

benar-benar tenang selama waktu tertentu sebelum membaca status tombol, atau melalui rangkaian elektronik menggunakan kombinasi resistor dan kapasitor.

1.2.6 Analog-to-Digital Converter

Banyak sensor di dunia nyata menghasilkan nilai yang terus berubah secara halus, seperti sensor cahaya LDR atau sensor suhu. Karena mikrokontroler hanya memahami angka biner, periferal ADC digunakan untuk mengubah tegangan analog tersebut menjadi angka digital.

ADC bekerja melalui tiga langkah: mengambil sampel tegangan sesaat, membagi tegangan tersebut ke tingkatan angka terdekat, dan mengubahnya menjadi kode biner. ESP32 memiliki ADC dengan ketelitian 12-bit yang membagi rentang tegangan 0 sampai 3.3V menjadi 4096 tingkatan angka, yaitu dari nilai 0 hingga 4095.

2 Tugas Pendahuluan

Soal 1

Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

Jawaban: Perbedaan utama antara framework Arduino dan ESP-IDF terletak pada cara kerja penulisan program, tingkat kemudahan perintah, pengaturan tugas kerja sistem, dan efisiensi memori.

1. **Struktur Program dan Alur Kerja:** Pada framework Arduino, alur program dibuat sangat sederhana dengan dua fungsi utama, yaitu `setup` yang dijalankan satu kali saat alat baru menyala untuk persiapan awal, dan `loop` yang dijalankan terus-menerus secara berurutan. Sebaliknya, pada ESP-IDF, program dibangun menggunakan sistem operasi FreeRTOS. Titik awal program dimulai dari fungsi `app_main`. Dari fungsi ini, pembuat program dapat membuat beberapa tugas berbeda yang bisa berjalan bersamaan dan mengatur prioritasnya pada inti prosesor yang diinginkan.

2. **Tingkat Kemudahan dan Akses Perangkat Keras:** Arduino menyediakan perintah-perintah praktis seperti `pinMode`, `digitalWrite`, dan `delay`. Perintah ini sengaja dibuat sederhana agar kode yang sama bisa dipakai di berbagai board mikrokontroler yang berbeda. Namun, penyederhanaan ini membuat beberapa fitur canggih bawaan chip menjadi tersembunyi. Sementara itu, ESP-IDF merupakan framework bawaan asli dari pabrikan chip. Kode program pada ESP-IDF ditulis lebih detail dan langsung mengakses komponen internal chip, seperti manajemen daya hemat baterai dan pengaturan jaringan Wi-Fi tingkat lanjut.

3. **Pengaturan Waktu dan Penundaan:** Perintah `delay` pada Arduino membuat prosesor berhenti bekerja dan hanya menunggu sampai waktu selesai, sehingga prosesor tidak bisa mengerjakan tugas lain pada saat itu. Pada ESP-IDF, penundaan waktu menggunakan perintah `vTaskDelay`. Perintah ini hanya mengistirahatkan tugas yang bersangkutan, sementara tugas lain yang sedang mengantre tetap bisa dikerjakan oleh prosesor.

Tabel 1: Perbandingan Framework Arduino dan ESP-IDF

Hal yang Dibandingkan	Framework Arduino	Framework ESP-IDF
Titik Awal Program	Fungsi `setup` dan `loop`	Fungsi utama `app_main`
Tingkat Kemudahan Kode	Sangat mudah dipahami dan sederhana	Lebih rumit dan butuh pemahaman sistem
Dukungan Multi-tugas	Terbatas dan berjalan di balik layar	Sangat lengkap menggunakan FreeRTOS
Penggunaan Memori	Cenderung lebih besar karena library pembungkus	Sangat hemat dan efisien
Kurva Belajar	Cepat dipelajari oleh pemula	Butuh waktu belajar lebih lama
Tujuan Penggunaan	Proyek hobi, belajar, dan prototipe cepat	Produk komersial dan perangkat industri

Soal 2

Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk di-flash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.

Jawaban: Penyebab utama dari perbedaan cara mengisi program ini adalah ada atau tidaknya chip perantara USB-to-Serial pada papan sirkuit masing-masing board.

1. **Board ESP32:** Pada board ESP32 DevKit, sudah ada chip perantara USB-to-Serial di papannya, seperti CP2102 atau CH340. Chip ini bertugas menerjemahkan data dari port USB komputer menjadi sinyal serial UART yang dipahami oleh pin GPIO1 dan GPIO3 pada chip ESP32. Selain itu, chip ini memiliki rangkaian otomatis yang bisa mereset dan menarik pin bootloader ESP32 ke kondisi siap menerima program secara otomatis saat proses upload dimulai dari komputer. Karena semua rangkaian perantara sudah terpasang di papannya, kita cukup mencolokkan kabel USB biasa untuk mengisi program.
2. **Board STM32 Blackpill:** Board STM32 Blackpill dibuat dengan desain yang kompak dan hemat biaya, sehingga di papannya tidak dipasang chip perantara USB-to-Serial. Port USB Type-C yang ada di board langsung tersambung ke pin USB internal mikrokontroler pada pin PA11 dan PA12. Untuk mengisi memori Flash dan memeriksa program langsung di dalam chip STM32, pabrikan menyediakan jalur khusus bernama Serial Wire Debug yang memerlukan pin SWDIO untuk data dan pin SWCLK untuk pewaktu. Komputer biasa tidak memiliki colokan yang bisa langsung berbicara dengan protokol ini. Oleh karena itu, kita membutuhkan alat perantara dari luar yaitu ST-Link V2. ST-Link bertugas menerima file program dari USB komputer lalu menyalurkannya ke pin debug STM32 tersebut.

Soal 3

Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.

Jawaban: Perbedaan logika pada kedua rangkaian ini ditentukan oleh ke mana arah pin masukan terhubung saat tombol sakelar sedang bebas atau sedang ditekan.

1. **Rangkaian Pull-Up:** Pada rangkaian ini, pin GPIO dihubungkan ke sumber tegangan 3.3V melalui resistor 10 kΩ, sedangkan tombol dipasang menghubungkan pin GPIO ke ground.
    - **Saat Tombol Tidak Ditekan:** Jalur ke ground terbuka. Karena tidak ada arus yang mengalir ke ground, tegangan pin GPIO ditarik menuju 3.3V oleh resistor. Akibatnya, pin membaca nilai logika HIGH.
    - **Saat Tombol Ditekan:** Tombol menutup dan menghubungkan pin GPIO langsung ke ground. Arus mengalir melewati resistor ke ground, sehingga tegangan pada pin turun menjadi 0V. Akibatnya, pin membaca nilai logika LOW. Cara kerja ini disebut logika active-low.

2. **Rangkaian Pull-Down:** Pada rangkaian ini, pin GPIO dihubungkan ke ground melalui resistor 10 kΩ, sedangkan tombol dipasang menghubungkan pin GPIO ke sumber tegangan 3.3V.

    - **Saat Tombol Tidak Ditekan:** Jalur ke tegangan 3.3V terbuka. Resistor menarik tegangan pin ke ground sehingga tegangannya 0V. Akibatnya, pin membaca nilai logika LOW.
    - **Saat Tombol Ditekan:** Tombol menutup dan mengalirkan arus dari 3.3V ke pin GPIO. Tegangan pada pin naik menjadi 3.3V. Akibatnya, pin membaca nilai logika HIGH. Cara kerja ini disebut logika active-high.

Tabel 2: Kondisi Logika Resistor Pull-Up dan Pull-Down

Rangkaian	Kondisi Tombol	Tegangan pada Pin	Logika Terbaca	Sifat Logika	
Pull-Up	Tidak Ditekan	Sekitar 3.3V	HIGH (1)	Kondisi diam	
Pull-Up	Ditekan	Sekitar 0.0V	LOW (0)	Kondisi aktif	
Pull-Down	Tidak Ditekan	Sekitar 0.0V	LOW (0)	Kondisi diam	
Pull-Down	Ditekan	Sekitar 3.3V	HIGH (1)	Kondisi aktif	

Soal 4

Jelaskan fenomena bouncing pada tombol mekanik, mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.

Jawaban:

1. **Penyebab Bouncing:** Bouncing adalah gangguan sinyal berupa perubahan cepat bolak-balik antara nilai HIGH dan LOW yang terjadi sesaat ketika kontak tombol mekanik ditekan atau dilepaskan. Gangguan ini biasanya berlangsung sangat singkat antara 1 sampai 50 milidetik.

    Hal ini terjadi karena kontak sakelar tombol terbuat dari pelat logam yang lentur. Ketika kita menekan tombol, pelat logam tersebut memantul-mantul kecil sebelum akhirnya menempel dengan rapat. Karena mikrokontroler membaca sinyal dengan kecepatan jutaan kali per detik, pantulan mekanis yang sangat cepat ini disangka sebagai penekanan tombol berulang-ulang, padahal kita hanya menekan tombol satu kali.

2. **Cara Mengatasinya:**

    - **Melalui Program:** Cara ini menggunakan perhitungan waktu di dalam kode program tanpa memerlukan komponen tambahan. Saat program mendeteksi adanya perubahan sinyal tombol, program mencatat waktu awal menggunakan fungsi penghitung waktu seperti `millis`. Program kemudian menunggu dan memeriksa apakah sinyal tombol tersebut tetap tenang dan tidak berubah selama waktu tertentu, misalnya 30 sampai 50 milidetik. Jika sinyal sudah benar-benar tenang, status penekanan tombol baru dianggap sah. Cara ini sangat hemat karena tidak memerlukan biaya komponen tambahan dan waktu tunggunya mudah diubah di kode.
    - **Melalui Rangkaian Elektronik:** Cara ini menyaring getaran langsung pada rangkaian sebelum sinyal masuk ke pin mikrokontroler. Rangkaian yang digunakan adalah perpaduan

resistor dan kapasitor yang dipasang dekat tombol. Kapasitor berfungsi meredam lonjakan listrik yang cepat dari pantulan pelat logam, sehingga tegangan yang keluar berubah secara halus. Untuk merapikan sinyal tersebut menjadi kotak digital yang bersih, keluarannya disambungkan ke IC Schmitt Trigger seperti 74HC14. Cara ini membuat prosesor mikrokontroler tidak terbebani oleh tugas pemfilteran getaran tombol.

Soal 5

Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.

Jawaban:

1. **Tahapan Mengubah Sinyal Analog Menjadi Digital:** Sinyal analog memiliki nilai tegangan yang terus berubah secara halus, sedangkan mikrokontroler hanya mengolah angka biner 0 dan 1. Periferal ADC mengubah sinyal ini melalui tiga tahapan:

   - **Pencuplikan:** Rangkaian penahan mengambil nilai tegangan analog pada satu titik waktu tertentu dan menahan nilainya agar tidak berubah selama proses pembacaan.
   - **Kuantisasi:** Nilai tegangan yang ditahan tersebut dicocokkan ke tingkatan angka terdekat yang tersedia di dalam sistem ADC. ESP32 menggunakan metode pencarian bertahap untuk mencocokkan tegangan input dengan tegangan referensi internal secara cepat.
   - **Pengodean:** Setelah tingkatan yang paling mendekati ditemukan, tingkatan tersebut diubah menjadi susunan angka biner digital.

   Rumus dasar konversi tegangan ke nilai digital adalah:

   $$\text{Nilai Digital} = \left\lfloor \frac{V_{\text{in}}}{V_{\text{ref}}} \times (2^n - 1) \right\rfloor \tag{1}$$

   Pada ESP32 dengan resolusi $n = 12$ bit dan tegangan referensi $V_{\text{ref}} = 3.3$ V, hasil pembacaan menghasilkan rentang angka dari 0 sampai 4095. Untuk menghitung kembali perkiraan tegangannya, kita menggunakan rumus:

   $$V_{\text{in}} = \text{Nilai Digital} \times \left(\frac{3.3 \text{ V}}{4095} \right) \tag{2}$$

2. **Pengaruh Resolusi Bit Terhadap Ketelitian Pembacaan:** Resolusi bit yang disimbolkan dengan n menentukan berapa banyak tingkatan angka pembagi yang dimiliki oleh ADC, yaitu sebanyak 2^n tingkatan dari 0 hingga $2^n - 1$.

   Perubahan tegangan terkecil yang bisa dirasakan oleh satu angka kenaikan nilai digital disebut ukuran langkah:

   $$\Delta V = \frac{V_{\text{ref}}}{2^n - 1} \tag{3}$$

   Pengaruh resolusi bit terhadap hasil pembacaan adalah:

- **Pembacaan Menjadi Lebih Halus:** Semakin besar nilai bit n, jumlah tingkatan pembaginya menjadi semakin banyak. Hal ini membuat nilai satu langkah tegangan ΔV menjadi semakin kecil, sehingga selisih pembulatan berkurang dan hasil digital menjadi semakin mirip dengan tegangan aslinya.
- **Mampu Mendeteksi Perubahan Kecil:** ADC dengan bit yang lebih tinggi mampu mendeteksi perubahan tegangan yang sangat halus dari sensor.

Tabel 3: Perbandingan Ketelitian Berdasarkan Resolusi Bit pada Tegangan 3.3V

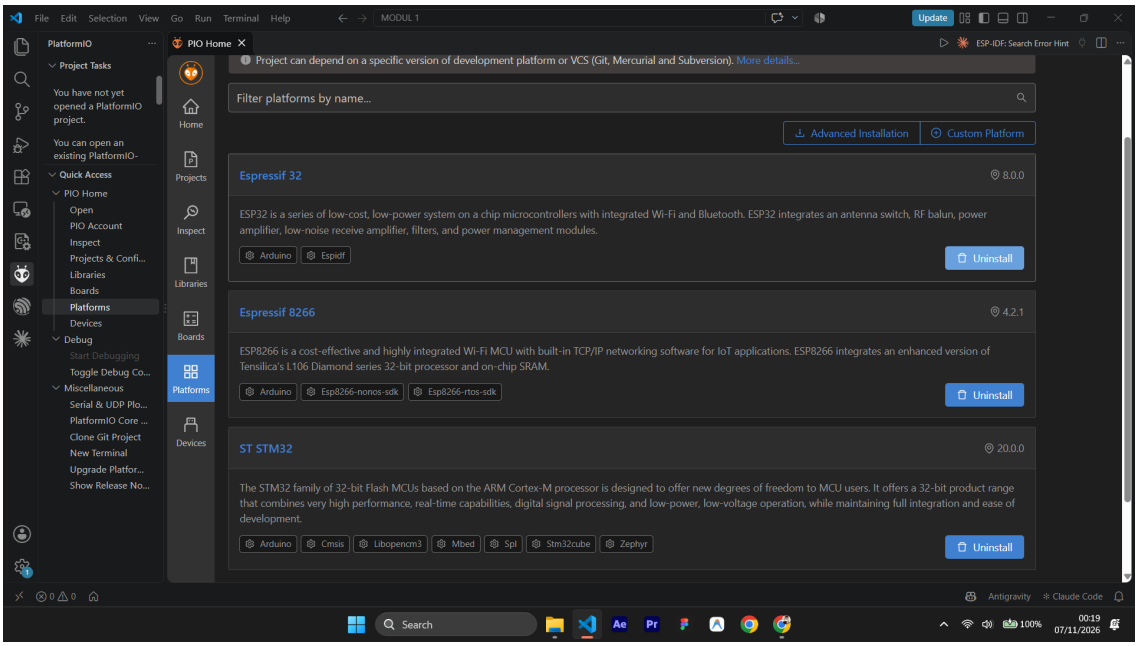
Resolusi Bit	Jumlah Tingkatan	Ukuran Langkah Tegangan	Tingkat Ketelitian
8-bit	256 tingkatan	12.94 mV per langkah	Kasar, hanya untuk deteksi sederhana
10-bit	1024 tingkatan	3.22 mV per langkah	Cukup, standar pada Arduino Uno
12-bit	4096 tingkatan	0.806 mV per langkah	Teliti, standar bawaan pada ESP32
16-bit	65536 tingkatan	0.050 mV per langkah	Sangat teliti untuk alat ukur presisi

Sebagai contoh, ADC 12-bit pada ESP32 memiliki ukuran langkah sekitar 0.806 mV. Artinya, kenaikan tegangan sekecil 0.8 milivolt sudah mampu dideteksi dan mengubah angka digital, sehingga jauh lebih teliti daripada ADC 10-bit.

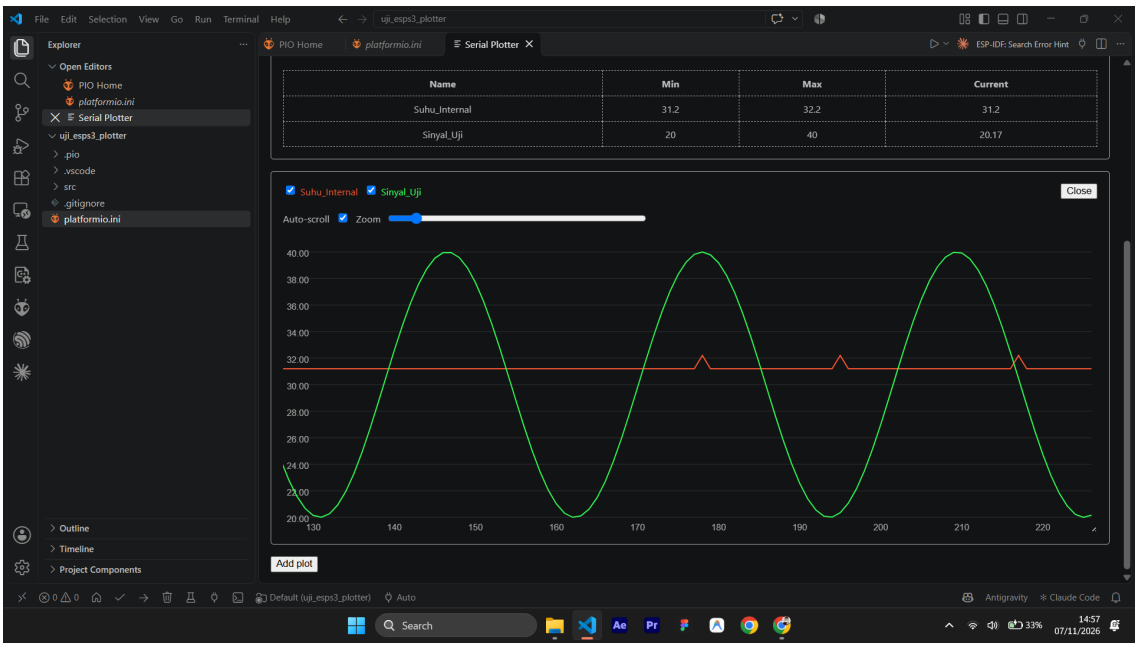
Soal 6

Lakukan set-up untuk tools: vscode, ekstensi platform-io (dengan framework esp-idf dan stm32), dan ekstensi serial-plotter pada laptop kalian, berikan screenshoot bahwa laptop kalian sudah ter-set-up dengan baik.

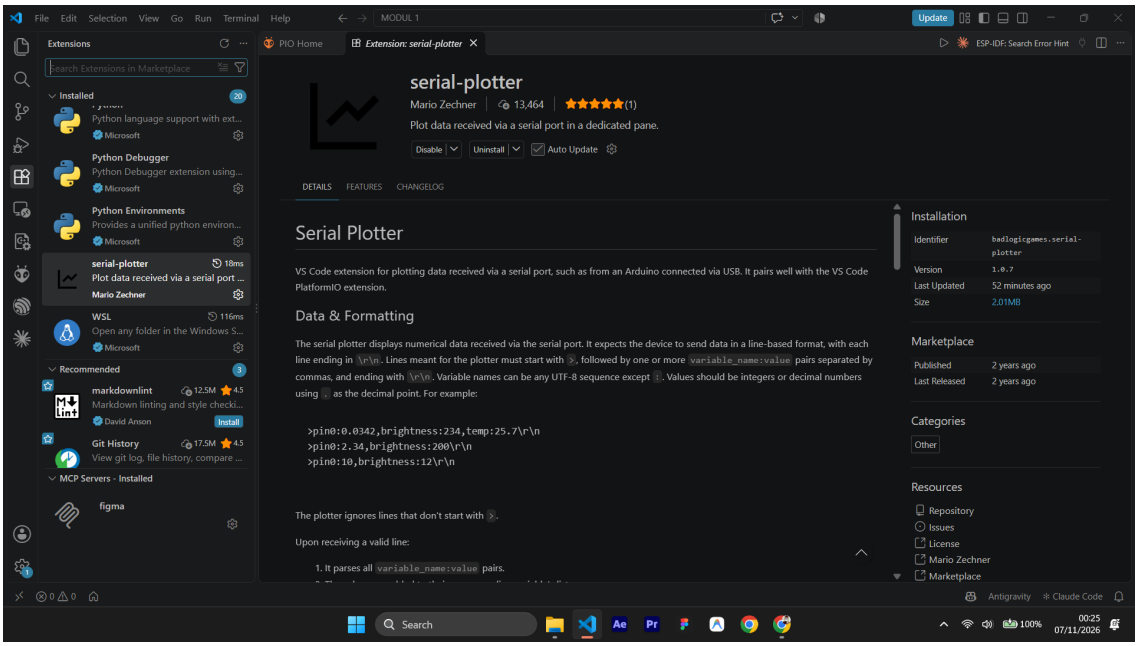
Jawaban: Berikut hasil screenshot set-up tools yang telah dilakukan:



Gambar 1: Bukti instalasi platform Espressif 32 dan ST STM32 pada PlatformIO IDE.



Gambar 2: Contoh percobaan pengecekan suhu internal pada Serial Plotter di Visual Studio Code.



Gambar 3: Tampilan antarmuka panel Serial Plotter yang telah siap digunakan.